

Scalable Technologies for Data Analysis

Data-oriented modeling

Davide Carneiro

Mestrado em Engenharia Informática

2025/2026

From the analytical problem to the pipeline

- By the end of this class you should be able to
 - Translate analytical questions into data products + metrics
 - Start thinking about deterministic DAGs (not yet implement them)
 - Define data contracts (schema/keys/semantics/SLOs)
 - Handle schema evolution + versioning
 - Map responsibilities across bronze/silver/gold

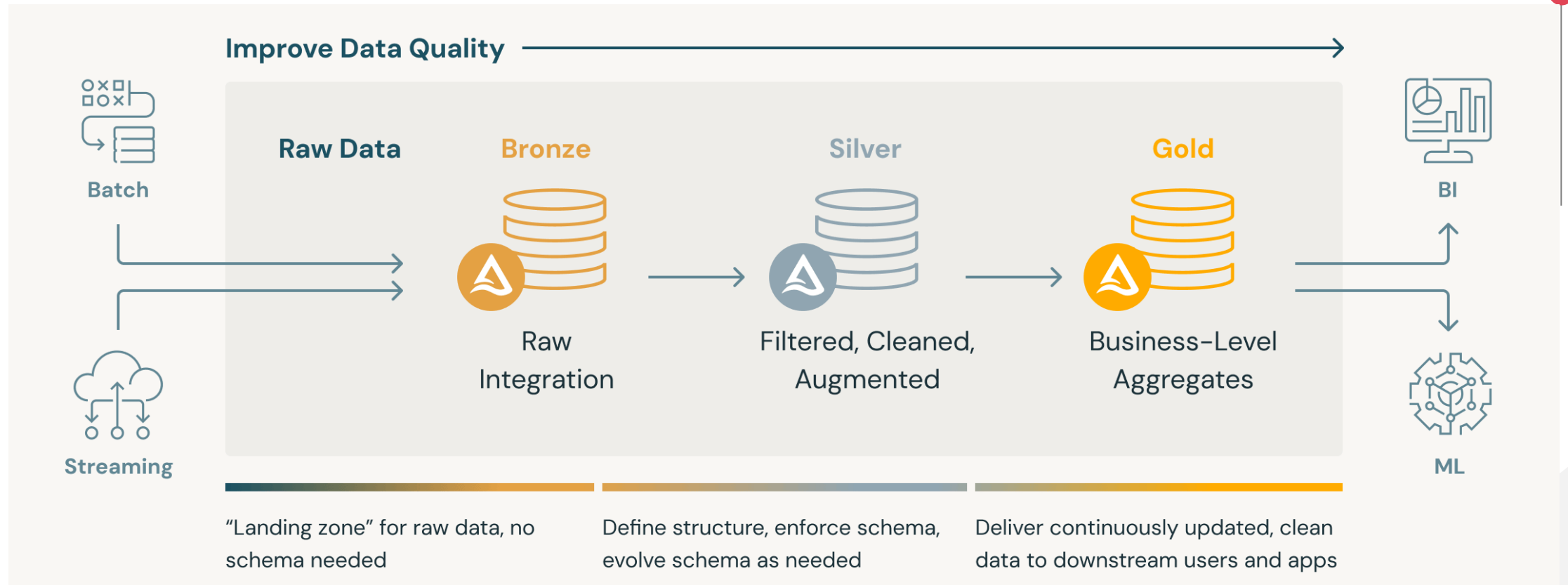
The plan

1. Intro to modern Data Platforms and scalability
2. Contract-Driven Design and Data Flow Modeling
3. Storage + table formats
4. Processing patterns + optimization
5. Orchestration + production reliability
6. Serving: SQL/APIs/semantic layer
7. Quality + observability + cost
8. Incremental + streaming
9. ML integration
10. Governance + evolution

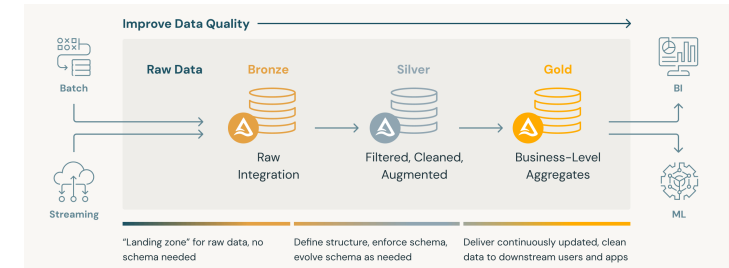


1. The basics

The Medallion Architecture



The Medallion Architecture



- A data design pattern for organizing a lake/lakehouse so that data quality and structure improve progressively from raw > refined > business-ready
- Everything we do **must be mapped** to one of these 3 stages

Bronze (raw / traceable)

- **Purpose:** ingest as-is, preserve fidelity
- **Typical ops:** minimal parsing, add metadata (ingest_time, source, batch_id)
- **Contract focus:** schema stability + provenance, not "perfect data"
- **Quality:** tolerant (nulls/duplicates allowed), but auditable
- **Usually append-only**, partition by event/ingest date

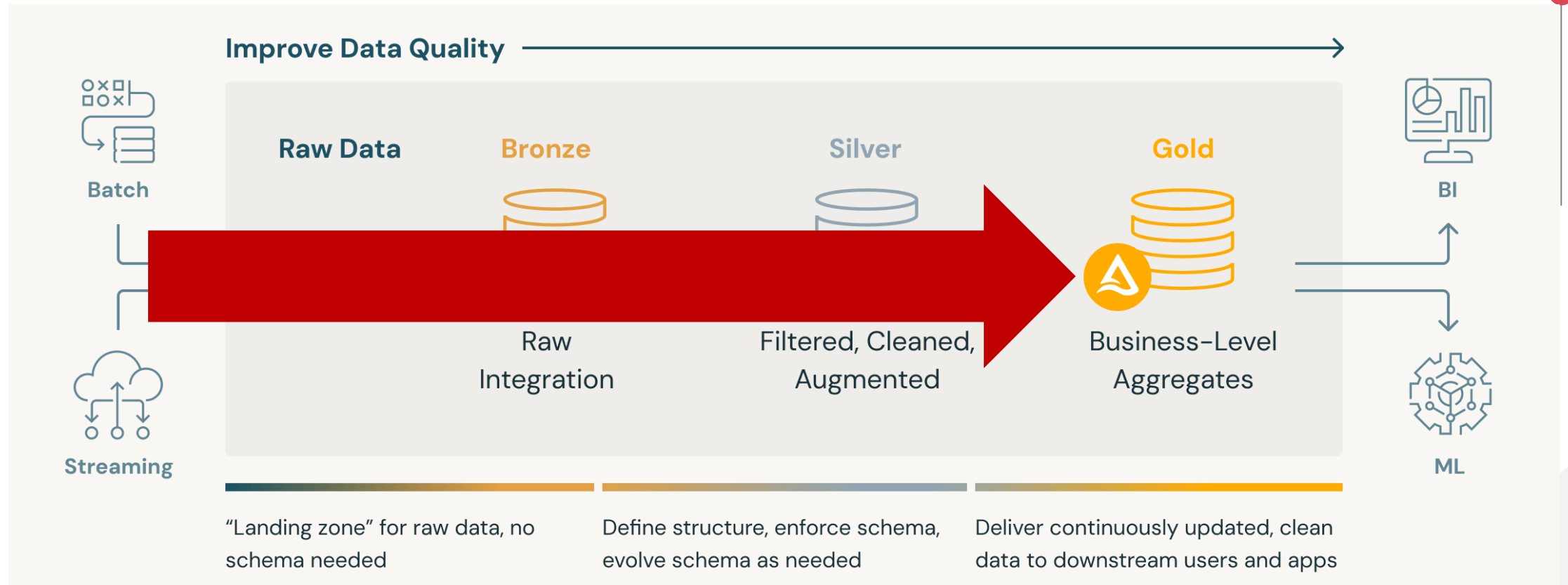
Silver (conformed / reliable)

- **Purpose:** make data usable and consistent
- **Typical ops:** cleaning, normalization, dedup, type fixes, standard timestamps/timezones
- **Contract focus:** keys + constraints + data quality
- **Quality:** stricter (uniqueness, ranges, nullability improves)

Gold (business / consumption)

- **Purpose:** serve metrics and decision-ready views
- **Typical ops:** aggregates, marts, KPI tables, feature-ready datasets
- **Contract focus:** business semantics + metric definitions + stability
- **Quality:** strictest + SLOs that matter to consumers

Why don't we take shortcuts?

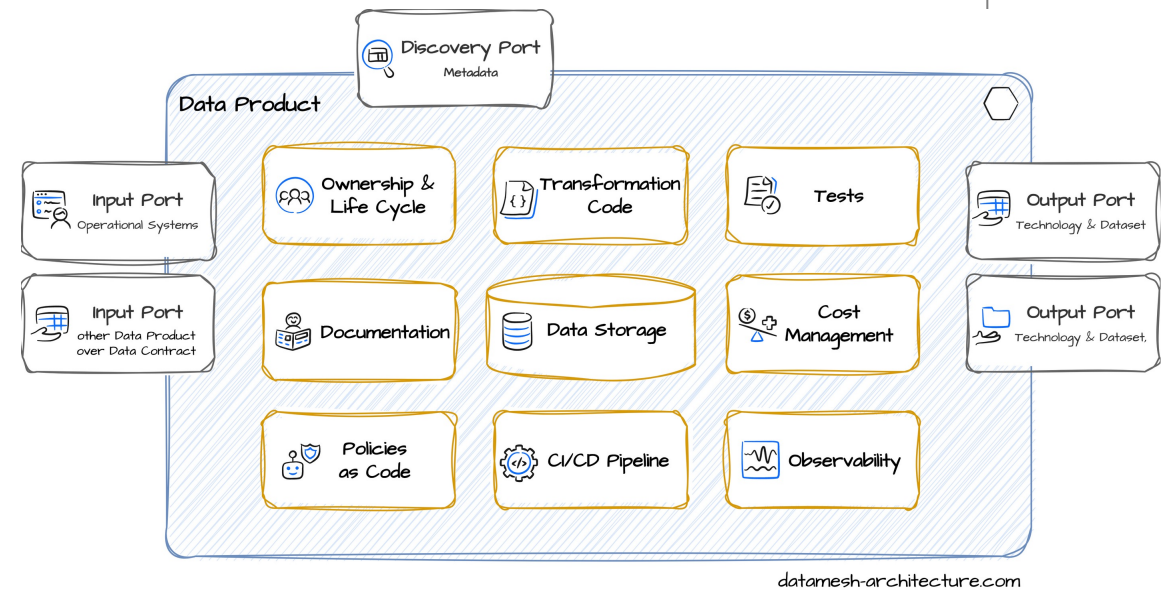


What is a Product?

What is a Data Product?

Data Product

- Is the actual thing/artifact we deliver **with an explicit purpose and consumers**
- It usually includes:
 - The dataset/table(s) (e.g. an Iceberg table in MinIO)
 - Documentation + ownership (who maintains it)
 - How it is produced (pipeline/workflow)
 - How it is consumed (SQL in Trino, features for ML, etc.)
 - Operational expectations (freshness, reliability)
 - Intended consumers
 - ...



Data Product

- Common elements, usually represented in some structured manner, include:
 - **product** (header): Describes what the data product is, its identity, purpose, owner, layer/domain, and who it serves
 - **delivery**: Specifies how the product is exposed and accessed, where it is stored (system/format/table), how it's updated, and how consumers query it (e.g. Trino tables/views)
 - **production**: Defines how the product is built and refreshed, the orchestration workflow, schedule, parameters (data intervals), and backfill capabilities
 - **lineage**: Records where the product comes from, its upstream sources and key transformation notes/dependencies for traceability and impact analysis
 - **observability**: States how we know the product is healthy, quality gates, dashboards, and alerts for freshness, completeness, failures, and regressions
 - **governance**: Captures the policy layer including data classification, PII posture, and access controls (who can read/write)
 - **references**: Points to the supporting artifacts including the contract file, docs, runbook, and any reproducibility requirements (e.g. MLflow logging conventions)

Data Product (example)

```
product:
  id: gold.customer_daily_kpis
  layer: gold
  domain: ecommerce
  owner:
    team: data-analytics
    contact: data-analytics@org
    on_call: data-analytics-oncall
  description: "Daily customer KPIs for BI dashboards and ML feature generation."
  consumers:
    - name: bi.conversion_dashboard
      type: bi
      access: trino_sql
    - name: ml.churn_model
      type: ml
      access: offline_features

delivery:
  storage:
    system: minio
    format: iceberg
    bucket: lakehouse
    table_identifier: lakehouse.gold.customer_daily_kpis # Trino/Iceberg identifier
    partitions:
      - date
    update_mode: merge # idempotent upsert by primary key
  serving:
    query_engine: trino
    stable_views:
      - lakehouse.semantic.customer_daily_kpis_v1 # optional compatibility/semantic view

production:
  orchestrator: flyte
  workflow: kpis.customer_daily_kpis_wf
  schedule: "0 2 * * *" # daily at 02:00
  parameters:
    - data_interval_start
    - data_interval_end
  backfills:
    supported: true
    unit: day
    max_lookback_days: 30
```

```
lineage:
  sources:
    - lakehouse.silver.orders_dedup
    - lakehouse.silver.payments_clean
  sla_dependencies:
    - upstream: lakehouse.silver.orders_dedup
      freshness_target: "P95 <= 1h"

observability:
  dashboards:
    - pipeline_health: observability.customer_daily_kpis
    - data_quality: quality.customer_daily_kpis
  alerts:
    - type: freshness_breach
      notify: "#data-alerts"
    - type: completeness_breach
      notify: "#data-alerts"

governance:
  data_classification: internal
  pii: none
  access_policy:
    read_roles:
      - analytics_readers
      - ml_readers
    write_roles:
      - data_platform_writers

references:
  contract_file: contracts/gold.customer_daily_kpis/contract.yaml
  runbook: runbooks/gold.customer_daily_kpis.md
  docs: docs/gold.customer_daily_kpis.md
```

Data Products vs. Medallion Architecture

- Data products may exist at Bronze, Silver, and Gold
 - It just depends on who the consumer is, and what level of guarantees they need
- Bronze data products (raw ingestion products)
 - **Consumers:** auditing, replay/reprocessing, debugging, compliance, sometimes power users
 - **Contract emphasis:** provenance + fidelity (“this is what we received”), stable ingestion semantics
 - **Example:** bronze.web_events_raw (Iceberg table in MinIO), append-only, includes source, ingest_time, original payload, minimal parsing
- Silver data products (conformed products)
 - **Consumers:** most downstream engineering/analytics pipelines, feature engineering, “trusted base tables”
 - **Contract emphasis:** keys, dedup rules, standardized timestamps/timezones, quality constraints
 - **Example:** silver.orders_dedup with a guaranteed unique key and normalized schema, used by multiple gold products
- Gold data products (business-facing products)
 - **Consumers:** BI dashboards, data apps, stakeholder reporting, ML training/serving features
 - **Contract emphasis:** business semantics + metric definitions + stability + stronger SLOs
 - **Example:** gold.customer_daily_kpis or gold.conversion_funnel_daily.

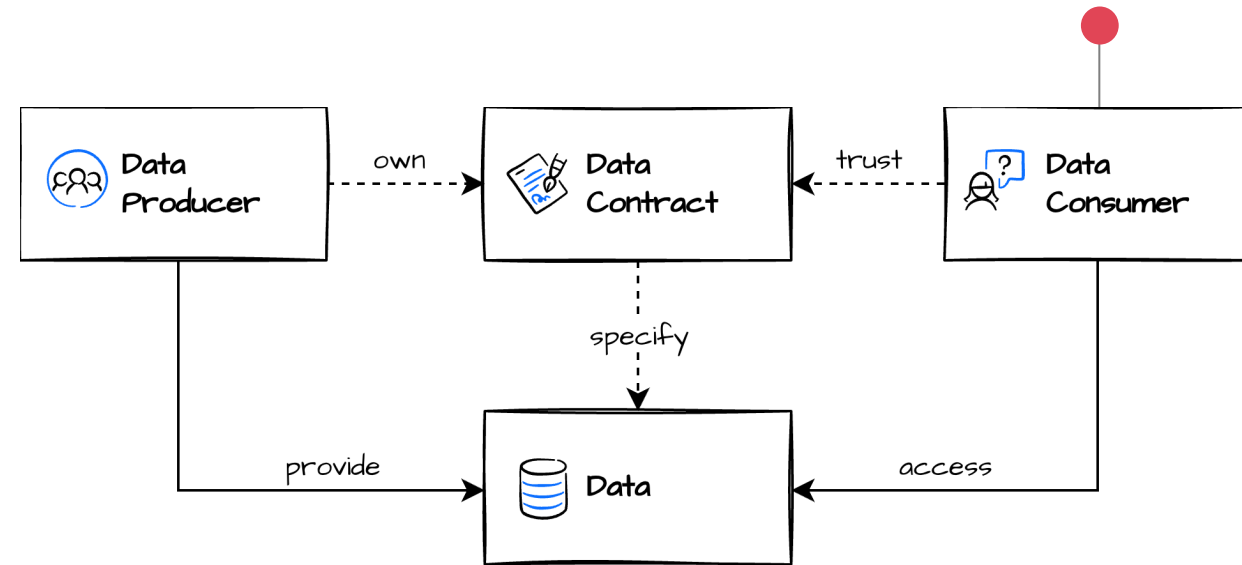


What is a **Contract?**

What is a **Data Contract?**

Data Contract

- A formal, versioned agreement between a producer (the pipeline/team that publishes a dataset) and its consumers (analytics, BI, downstream pipelines, ML) that defines the dataset as an interface
 - Similar an API contract, but for data
- It defines things like:
 - Grain, schema, keys, semantics
 - Constraints/quality expectations
 - SLOs (freshness/ completeness/ availability)
 - Compatibility + versioning rules



Data Contract

- Common elements, usually represented in some structured manner, include:
 - **contract** (header): Identifies the contract itself, including which dataset it applies to, its version/status, and the scope of the agreement between producer and consumers
 - **interface**: Defines the dataset's shape as an interface, especially the grain (what one row represents) and the primary key (how rows are uniquely identified)
 - **schema**: Specifies the field-level interface (e.g. column names, types, nullability, human-readable meanings) that consumers can rely on
 - **semantics**: States the business meaning and definitions behind fields/metrics (e.g. what "paid_order" or "revenue" means) to avoid multiple interpretations
 - **quality**: Declares the minimum acceptable quality bar, including constraints/checks (uniqueness, ranges, null rules) and the SLO targets (freshness, completeness, availability)
 - **behavior**: Defines how the dataset behaves over time (e.g. late data handling, correction strategy, determinism/reproducibility expectations)
 - **change management**: Establishes how the contract can evolve safely including compatibility rules, versioning bumps (SemVer), deprecation windows, and rollout strategies
 - **examples**: Provides concrete usage and samples, such as example queries and sample rows to help consumers validate their understanding quickly

Data Contract (example)

```
contract:
  id: gold.customer_daily_kpis
  version: 1.2.0
  applies_to_product: gold.customer_daily_kpis
  status: active

interface:
  grain: "1 row per (customer_id, date)"
  primary_key: ["customer_id", "date"]

schema:
  - name: customer_id
    type: string
    nullable: false
    description: "Stable customer identifier (surrogate id)."
  - name: date
    type: date
    nullable: false
    description: "Business date in UTC."
  - name: orders_count
    type: bigint
    nullable: false
    description: "Number of distinct paid orders on 'date'."
  - name: revenue_eur
    type: decimal(18,2)
    nullable: false
    description: "Sum of paid order totals in EUR (net of refunds applied up to T+7 days)."
  - name: is_active_30d
    type: boolean
    nullable: false
    description: "True if customer had >=1 paid order in the last 30 days ending on 'date'."

semantics:
  definitions:
    paid_order: "Order with successful payment and not cancelled."
    revenue_eur: "Net revenue after refunds recognized up to 7 days after purchase."
```

```
quality:
  constraints:
    - rule: "uniqueness(primary_key) == 100%"
      severity: error
    - rule: "orders_count >= 0"
      severity: error
    - rule: "revenue_eur >= 0"
      severity: error
    - rule: "no_nulls(customer_id, date, orders_count, revenue_eur, is_active_30d)"
      severity: error
  slo:
    freshness: "P95 <= 2h after end-of-day close"
    completeness: ">= 99.5% of expected customer-days present by T+2h"
    availability: ">= 99% monthly"
```

```
behavior:
  late_data_policy:
    allowed_lateness: "7 days"
    correction_strategy: "recompute affected partitions and MERGE by primary key"
  determinism:
    guarantees:
      - "same input interval + same code version + same source snapshots => same output"
    forbidden_patterns:
      - "reading unpinned 'latest' snapshots"
      - "non-deterministic now()/random seeds without control"
```

```
change_management:
  compatibility_rules:
    - change: "add_nullable_column"
      version_bump: minor
      compatibility: backward
    - change: "rename_column"
      version_bump: major
      compatibility: breaking
    - change: "drop_column"
      version_bump: major
      compatibility: breaking
    - change: "type_widening_int_to_bigint"
      version_bump: minor
      compatibility: backward
    - change: "type_narrowing_bigint_to_int"
      version_bump: major
      compatibility: breaking
  deprecation_window: "60 days"
  rollout_strategies:
    - "dual-write v1 and v2 for MAJOR changes"
    - "provide compatibility views during deprecation"
```

```
examples:
  consumer_queries:
    - "SELECT date, SUM(revenue_eur) FROM lakehouse.semantic.customer_daily_kpis_v1 GROUP BY 1;"
  sample_row:
    customer_id: "c_123"
    date: "2026-02-10"
    orders_count: 2
    revenue_eur: 59.90
    is_active_30d: true
```

Schema

- The schema is a central element in a Data Contract, defining not only the variables (e.g. name, type) but also their meaning
 - This should then be used by developers during implementation

```
from flytekit import task, workflow
import pandas as pd #This is a dependency of the StructuredDataset type
from flytekit import task, workflow

@task
def create_dataset() -> pd.DataFrame:
    # This will be stored as parquet by default
    return pd.DataFrame({
        "name": ["Alice", "Bob"],
        "age": [25, 30]
    })

@task
def process_dataset(df: pd.DataFrame) -> pd.DataFrame:
    # This task can accept the dataset because the formats match
    return df[df["age"] > 25]

@workflow
def my_workflow()-> pd.DataFrame:
    df = create_dataset()
    processed = process_dataset(df=df)
    return processed

message StructuredDatasetType {
    columns = [
        { name: "name", literal_type: { simple: STRING } },
        { name: "age", literal_type: { simple: INTEGER } }
    ]
    format = "parquet"
}
```

```
from flytekit import task, workflow
import pandas as pd

@task
def create_csv_dataset() -> pd.DataFrame:
    # Let's say we explicitly set this to store as CSV
    df = pd.DataFrame({"name": ["Alice"], "age": [25]})
    return df.with_format("csv")

@task
def process_parquet_dataset(df: pd.DataFrame) -> pd.DataFrame:
    #This task expects parquet format
    df = df.with_format("parquet")
    return df

@workflow
def my_workflow()-> pd.DataFrame:
    df = create_csv_dataset()
    # This would fail type checking because formats don't match
    # CSV != Parquet
    processed = process_parquet_dataset(df=df) # Type error!
    return processed
```

Data Product vs. Data Contract

- A product might have more than one contract, depending on how it's consumed (although it's usually all together):
 - A physical table contract (schema/keys/grain + SLOs)
 - A semantic contract (metric definitions, business logic) often materialized as a Trino view (in our case) or a metric layer
- A data contract is typically a core part of a data product, because it's the explicit promise/interface that makes the product safe to depend on

```
product:
  id: gold.customer_daily_kpis
  layer: gold
  domain: ecommerce
  owner:
    team: data-analytics
    contact: data-analytics@org
    on_call: data-analytics-oncall
  description: "Daily customer KPIs for BI dashboards and ML feature generation."
  consumers:
    - name: bi.conversion_dashboard
      type: bi
      access: trino_sql
    - name: ml.churn_model
      type: ml
      access: offline_features
```

```
delivery:
  storage:
    system: minio
    format: iceberg
    bucket: lakehouse
```

```
contract:
  id: gold.customer_daily_kpis
  version: 1.2.0
  applies_to_product: gold.customer_daily_kpis
  status: active

interface:
  grain: "1 row per (customer_id, date)"
  primary_key: ["customer_id", "date"]

schema:
  - name: customer_id
    type: string
    nullable: false
    description: "Stable customer identifier (surrogate id)."
```

```
  - name: date
    type: date
    nullable: false
    description: "Business date in UTC."
  - name: orders_count
    type: bigint
    nullable: false
    description: "Number of distinct paid orders on 'date'."
```

Data Contract vs. Medallion Architecture

- Contract in bronze
 - Focus on fidelity and metadata (source, ingest time)
- Contract in silver
 - Focus on data quality and consistency
- Contract in gold
 - Focus on business semantics and metric stability
- Golden rules:
 - Bronze is (ideally) immutable
 - Gold is not a dumping ground, it's a clean product

Typical problems without Data Contracts

- (Not so...) Silent breaking changes
 - Inconsistent metrics (“two truths”)
 - Fragile pipelines / painful backfills
 - ...
-
- The main goal of using data contracts is to reduce entropy (by controlling change) and ultimately **increasing trust**

2. From data pain to data products

From data pain to data products

Mostly Business

1. Identify the business problem and “data pains”
 - What decision is blocked or risky?
 - What’s failing today (inconsistent numbers, fragile pipelines, slow backfills, unclear definitions)?
 - Output: a short “pain statement” + who is affected (BI, Ops, ML, Finance).
2. Translate into analytical questions
 - Make them decision-oriented, not SQL:
 - “Did conversion drop? Where in the funnel?”
 - “Which customers are becoming inactive?”
 - Output: 2–5 questions with named consumers
3. Define metrics (operationally)
 - For each question, define the metrics including grain, population, formula, time rules, dedup/identity, late-data policy
 - Output: metric specs that remove ambiguity

4. Decide the product shape (grain + datasets)
 - Design the actual deliverable including: what table(s)/view(s) will we publish? What is the grain? What are the keys? What belongs in Bronze vs Silver vs Gold?
 - Output: a proposed product (e.g. gold.customer_daily_kpis) and the supporting silver foundations
5. Write the data contract
 - Turn the metric + dataset decisions into a consumer-facing promise that includes schema + semantics + constraints + SLOs + evolution rules
 - Output: contract.yaml (versioned)
6. Design the pipeline as a deterministic DAG
 - Bronze > silver > gold steps, explicit data intervals, idempotent writes/merges
 - Quality gates before publishing gold
 - Output: DAG sketch + “where checks happen”
7. Operationalize (SLOs, observability, reproducibility)
 - Dashboards/alerts, runbooks
 - Iceberg snapshots for reproducibility
 - MLflow logs dataset snapshot + contract version for training
 - Output: product.yaml + monitoring + ML reproducibility policy.

Mostly Technical

From data pain to data products

- Non-technical steps are often the hardest. Some examples
 1. Identifying “data pains” (symptoms in the organization)
 - “Two truths”: BI dashboard and finance report don’t match for “revenue”
 - Silent breaking changes: a column rename drops a KPI with no error
 - Duplicates / retries: orders suddenly double after a pipeline rerun
 - Late data: yesterday’s numbers keep changing for a week
 - Slow + fragile backfills: recomputing last month is painful and inconsistent
 2. Defining analytical questions (decision needs, not SQL queries or processing pipelines)
 - Growth: “Did conversion drop yesterday vs the previous 7 days? Where in the funnel?”
 - Ops: “Which regions have delivery delays above SLA this week?”
 - Product: “Do new users who complete onboarding in 24h retain better at D7?”
 - Finance: “What is net revenue by day after refunds, and how does it trend?”
 - ML: “Which customers show declining purchase frequency in the last 30 days?”
 3. Defining metrics
 - Conversion rate = purchasers / checkout_started, UTC day, exclude test traffic, dedup by event_id
 - Active customer (30d) = ≥ 1 paid purchase in last 30 days ending on date (inclusive), timezone = UTC
 - Net revenue (EUR) = sum(paid orders) - refunds recognized up to T+7 days, exclude cancelled orders
 - On-time delivery % = deliveries with delivered_at $\leq$ promised_at, per order-day, exclude manual overrides
 - Churn label = no purchase in next 60 days after reference date, defined with point-in-time correctness

Metric

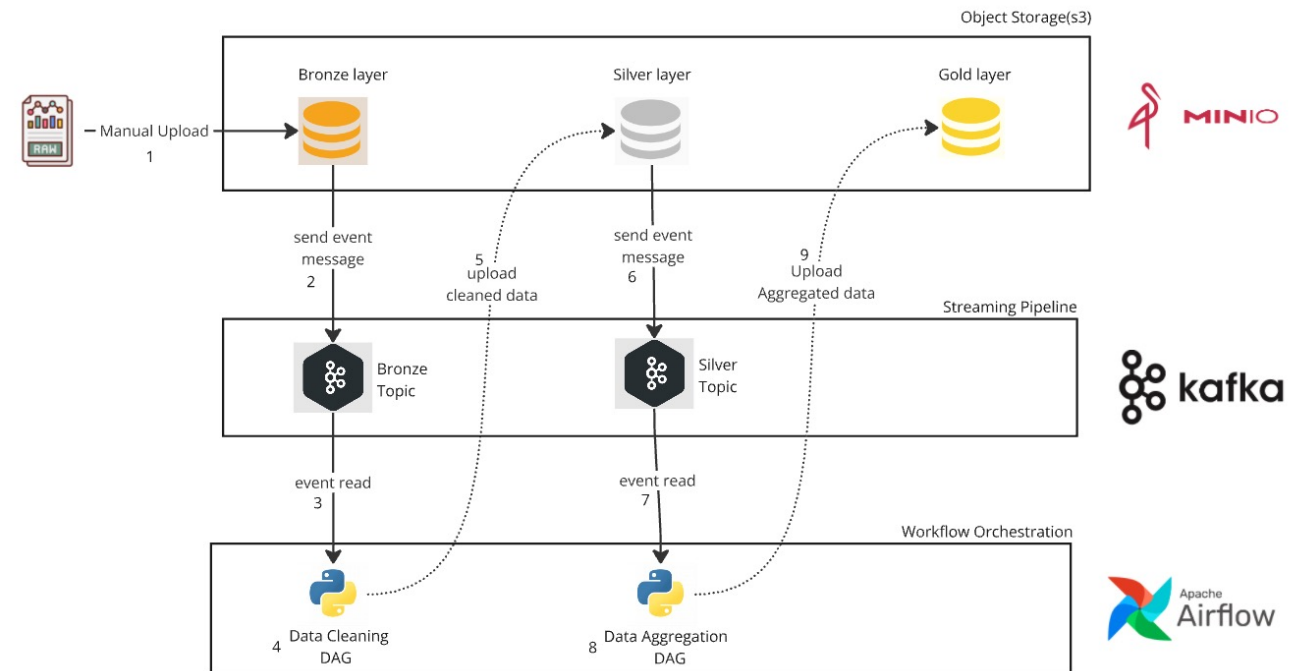
- A named, reusable measurement that answers a business question and that consumers can rely on consistently for dashboards, reports, alerts or ML features
- It is a contractable calculation, not just a number in a chart
- Metrics (with a more precise definition and rules, if necessary)
 - Net revenue (EUR) per day
 - Active customers (30d) (with a clear activity definition and window)
- Not Metrics
 - Revenue (without saying if paid vs invoiced, refund handling, timezone, grain, etc.)
 - Active users (with no population/window definition)

The anatomy of a Metric

- **Name + intent:** A stable name and what decision it supports
 - Example: `net_revenue_eur_daily` (tracks business performance day-by-day)
- **Grain (unit of measurement):** What entity/time level it's computed at
 - Example: "1 value per day" or "per customer-day" or "per order"
- **Population (who/what is included):** Inclusion/exclusion rules
 - Example: "paid orders, excluding cancelled and test orders"
- **Formula / logic:** Clear calculation
 - Example: `sum(order_total_eur) - sum(refunds_eur recognized up to T+7)`
- **Time definition:** Event time vs processing time, windowing, timezone, day boundary
 - Example: "group by UTC day of purchase_time"
- **Identity and dedup rules:** How you avoid double counting
 - Example: "deduplicate purchases by (source, event_id); keep earliest ingest"
- **Late data and corrections policy:** Whether the metric can change after publication
 - Example: "late events allowed up to 7 days; recompute affected partitions"
- **Quality expectations (optional but ideal):** Constraints or thresholds
 - Example: "net revenue must be ≥ 0 ; completeness $\geq 99.5\%$ by T+2h"
- **Versioning / change policy:** What constitutes a breaking change
 - Example: "changing refund window from 7 to 14 days is MAJOR"

Production

- Directed Acyclic Graph (DAG)
 - Nodes: tasks/steps that transform data
 - Edges: data dependencies (A must finish before B)
 - Acyclic: no loops, clear order, easier reasoning and recovery
- bronze.web_events_raw → silver.events_clean → gold.customer_daily_kpis
(Each arrow means “depends on output of the previous step”.)



Why move from scripts to DAGs?

- Step isolation: failures are contained, debugging is easier
- Partial recomputation: rerun only what changed/failed (e.g. recompute one day/partition)
- Observability and lineage: see what ran, with what inputs, and what it produced
- Common anti-patterns
 - Monolithic script: one huge job doing everything. It is hard to test, hard to rerun partially
 - Implicit dependencies: “magic” files, manual steps, shared folders. Makes code brittle and non-reproducible
- A data product **is no longer a script**, as it was before. It is a reproducible graph of well-defined steps

```
from math import sqrt
from flytekit import workflow
```

```
@task
```

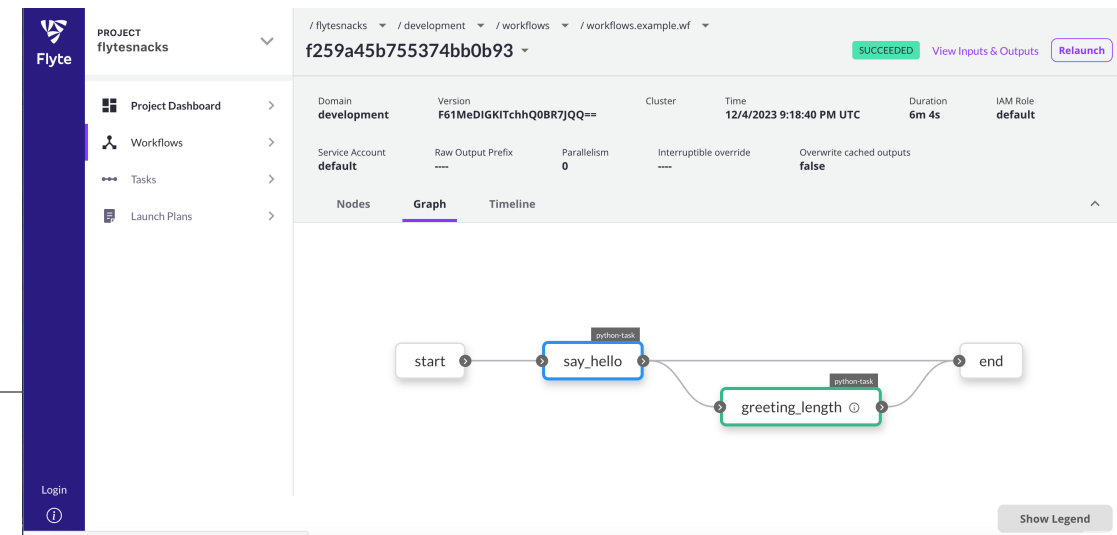
```
def standard_deviation(values: List[float], mu: float) -> float:
    variance = sum([(x - mu) ** 2 for x in values])
    return sqrt(variance)
```

```
@task
```

```
def standard_scale(values: List[float], mu: float, sigma: float) -> List[float]:
    return [(x - mu) / sigma for x in values]
```

```
@workflow
```

```
def standard_scale_workflow(values: List[float]) -> List[float]:
    mu = mean(values=values)
    sigma = standard_deviation(values=values, mu=mu)
    return standard_scale(values=values, mu=mu, sigma=sigma)
```



Determinism and reproducibility

- Determinism means getting the same output from the same input and same version
- Common sources of non-determinism
 - `now()` and system clock
 - out-of-order data arrivals
 - uncontrolled random seeds
 - reading “latest” without pinning a version/snapshot
- Practical translation to our stack (to be detailed later)
 - Flyte: explicit inputs, versions, traceable executions
 - Iceberg: snapshots/time travel for controlled reprocessing

“Time” in pipelines: run time vs. data time

- Run time: execution time (when the DAG ran)
- Data time: the interval/event time of the data, that is, which period the data refers to
- Implications (to be detailed later)
 - **Date-based partitioning** – Organizing a dataset into separate chunks (partitions) by a date field (e.g. event_date or date) so queries and pipelines can read/write only the relevant days efficiently
 - **Backfills and recomputation** – Re-running pipeline logic for a past time interval to (re)produce correct outputs, typically to incorporate late data, fix bugs, or apply changed logic, often by recomputing affected date partitions only
 - **Idempotency** – Designing a pipeline step so that running it multiple times with the same inputs produces the same end state. For example, no double-counting or duplicated outputs
 - **Retries**: Automatically re-executing a failed step (e.g. after a transient error) to recover without manual intervention, which should be made safe by idempotent design

Granularity

- The level of granularity must be explicit in the contract
- It must also fit the business question
 - Too coarse, we don't have the necessary "resolution". Example: aggregating by month when we needed a daily KPI
 - Too fine, we are storing/providing excessive data/detail (that might have to be processed later). Example: Storing the whole address when our KPI is by country
 - It also tends to add PII / governance issues if in Gold
 - Not so problematic (and often encouraged) if in Bronze
- Caveat: this depends on whether bronze / silver / gold
- Common mistakes
 - Aggregating too early / too late
 - Mixing grains in the same dataset



COARSE



FINE

What does one row means?

- **Keys**, define identity (“what is the same thing?”)

- Natural key

- Comes from the source domain
 - Examples: `order_id`, `invoice_number`
 - Pros: meaningful, often stable in the source

- Cons: may change (business identifiers sometimes get corrected, re-assigned, merged, or reformatted over time) may not exist, can be messy across systems

- Surrogate key

- Generated identifier (internal)
 - Examples: `customer_id` (UUID), `user_sk` (hash)
 - Pros: stable inside the platform, good for joins, hides PII
 - Cons: requires mapping and can break if the mapping changes

- **Event** identity identifies specific and unique events

- In event streams, `event_id` alone is often not enough (collisions across sources)
 - A common robust (minimal) pattern is a composite key: (`source_system`, `event_id`)

interface:

```
grain: "1 row per (customer_id, date)"
primary_key: ["customer_id", "date"]
```

Deduplication: when and where?

- Bronze (raw): usually no dedup
 - Keep “as-is”, even for audit/replay (it may help identify sources of data pains)
 - Preserve ingestion evidence
- Silver (clean/conformed):
 - Dedup happens here most of the time
 - We commit to a rule and produce a “trusted” dataset
- Gold (business)
 - Ideally consumes already deduped Silver
 - Gold focuses on business semantics/aggregations, not raw cleanup

Fidelity vs. Cost tradeoff (regarding Granularity and Dedup)

- In several stages, including when deciding about granularity and deduplication, there is a fidelity vs. cost tradeoff
- More fidelity (keep all duplicates + raw payloads)
 - Improves auditability, replay, debugging
 - Has a cost in terms of higher storage, heavier downstream processing
- More cleanup earlier (aggressive dedup)
 - Makes downstream tasks cheaper/faster, usually more stable KPIs (less margin for error)
 - Increases the risk of permanently deleting “real” records if the rule is wrong
- Often, in the early stages of maturity, we optimize for fidelity. When pipelines and business needs stabilize, we optimize for storage / processing
 - There’s always the risk of losing detail that will be needed later...

SLAs vs SLOs

- SLA (Service Level Agreement), external promise
 - A commitment made to consumers/stakeholders about service quality
 - Often tied to expectations, escalation, or penalties (formal or informal)
 - Example: “Daily KPIs available by 09:00 every day” (used by leadership meeting)
- SLO (Service Level Objective), internal operational target
 - An engineering goal the team uses to operate and improve the pipeline
 - Usually stricter than the SLA (buffer for incidents)
 - Example: “P95 freshness $\leq$ 2h after end-of-day close (30d)” so the 09:00 SLA is safe
 - Which means: Over a period of observation (last 30 days), at least 95% of days had the data available up until the last 2 hours, by the day ending
- Typical measures
 - Freshness (latency), e.g. “How old is the data when I query it?”
 - Completeness, e.g. “How much of the expected data arrived and passed checks?”
 - Availability, e.g. “Can consumers access it when they need it?”
 - Correctness / quality / accuracy, e.g. “<1% of data on client daily KPIs is missing”

In practice...

- Give visibility to SLOs by connecting pipelines to dashboards, and track SLO metrics over time
 - Last successful run time
 - Freshness distribution (P50/P95)
 - Completeness % per partition/day
 - Data quality failures by rule
- This helps answer: “Is today’s data safe to use?”
- Then define **quality gates** before publishing Gold (trust)
 - Don’t publish “gold” unless it meets minimum quality.
 - Typical gates:
 - Schema matches contract (types/nullability)
 - Key uniqueness holds
 - Critical fields not null
 - Completeness above threshold
 - Pass, publish gold
 - Fail, block publication, alert, and/or publish with a visible “degraded” flag (depending on policy)

```
quality:
  constraints:
    - rule: "uniqueness(primary_key) == 100%"
      severity: error
    - rule: "orders_count >= 0"
      severity: error
    - rule: "revenue_eur >= 0"
      severity: error
    - rule: "no_nulls(customer_id, date, orders_count, revenue_eur, is_active_30d)"
      severity: error
  slo:
    freshness: "P95 <= 2h after end-of-day close"
    completeness: ">= 99.5% of expected customer-days present by T+2h"
    availability: ">= 99% monthly"
```

Schema evolution

- Over time, there might be changes to the schema of a data product, each with a different impact
 - Add column (usually compatible)
 - Rename (often breaking for consumers unless we provide a compatibility layer (e.g., view/alias))
 - Type change (risky)
 - Widening (int > bigint, float > double): sometimes compatible
 - Narrowing (bigint > int, decimal > int): usually breaking / risky
 - Semantic type change (string > timestamp): high risk
 - Drop column (almost always breaking)
- The data contract should include compatibility rules
 - Backward-compatible – old queries/jobs keep running after the producer updates
 - Forward – consumers written for the new schema can still read data produced with the old schema
 - Full – old consumers work with new schema and new consumers work with old schema (often by adding fields in a safe way and avoiding removals/renames, or by using compatibility views)
 - Breaking
 - Technical – at least one existing consumer will fail (e.g. dashboard, ML)
 - Semantic – this still run, but are wrong (e.g. change salary from net to total, change timezone)

```
change_management:
  compatibility_rules:
    - change: "add_nullable_column"
      version_bump: minor
      compatibility: backward
    - change: "rename_column"
      version_bump: major
      compatibility: breaking
    - change: "drop_column"
      version_bump: major
      compatibility: breaking
    - change: "type_widening_int_to_bigint"
      version_bump: minor
      compatibility: backward
    - change: "type_narrowing_bigint_to_int"
      version_bump: major
      compatibility: breaking
  deprecation_window: "60 days"
  rollout_strategies:
    - "dual-write v1 and v2 for MAJOR changes"
    - "provide compatibility views during deprecation"
```

Contract versioning and rollout strategy

- What to do when a breaking (or eventually other) change must be done
 - Bump MAJOR contract version (e.g. 1.2.0 -> 2.0.0)
 - MAJOR: breaking change (consumers must change)
 - MINOR: backward-compatible change (old consumers still work)
 - PATCH: bugfix/docs/clarification that doesn't change the interface
 - Provide a compatibility view (e.g. in Trino) or dual-write old+new for a deprecation window
 - Communicate a cutoff date and migration steps

```
change_management:
  compatibility_rules:
    - change: "add_nullable_column"
      version_bump: minor
      compatibility: backward
    - change: "rename_column"
      version_bump: major
      compatibility: breaking
    - change: "drop_column"
      version_bump: major
      compatibility: breaking
    - change: "type_widening_int_to_bigint"
      version_bump: minor
      compatibility: backward
    - change: "type_narrowing_bigint_to_int"
      version_bump: major
      compatibility: breaking
  deprecation_window: "60 days"
  rollout_strategies:
    - "dual-write v1 and v2 for MAJOR changes"
    - "provide compatibility views during deprecation"
```

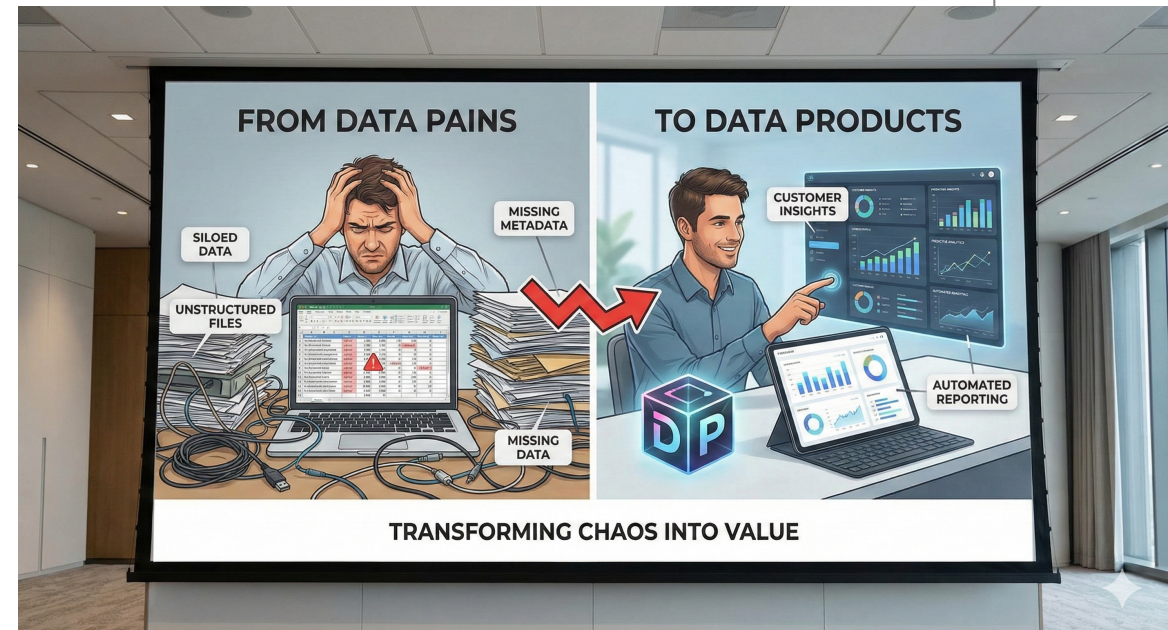
How a technological stack helps

- Iceberg (table format) makes “reliable tables” possible
 - Supports schema evolution: add columns / evolve schema in a controlled way (less brittle pipelines)
 - Snapshots > reproducibility: time travel to a known table state for backfills, debugging, and ML reproducible training
 - Writer/reader isolation: writers can commit new data while readers keep seeing a consistent table state (fewer “half-written” reads)
- Trino (query/serving layer) makes stable consumption practical
 - Consistent reads (when applicable): can read a table at a stable point-in-time (often aligned with Iceberg snapshots)
 - Views to abstract change: create semantic/compatibility views so consumers keep a stable interface while underlying tables evolve
- Iceberg stabilizes storage + history, Trino stabilizes access + interfaces

Hands-on: from data pain to data products

You have a messy dataset (raw event logs) and must turn it into a reliable data product that can be used by both BI (dashboards) and ML (feature generation) teams

The point is not to write code (yet) it's to practice thinking like data engineers/ analytics engineers: define meaning, control change, and design a pipeline that can be operated in production



Scalable Technologies for Data Analysis

Data-oriented modeling

Davide Carneiro

Mestrado em Engenharia Informática

2025/2026